
TESTING DURANTE CICLO DE VIDA DEL DESARROLLO DE SOFTWARE

Construcción del Software



PRESUPUESTO



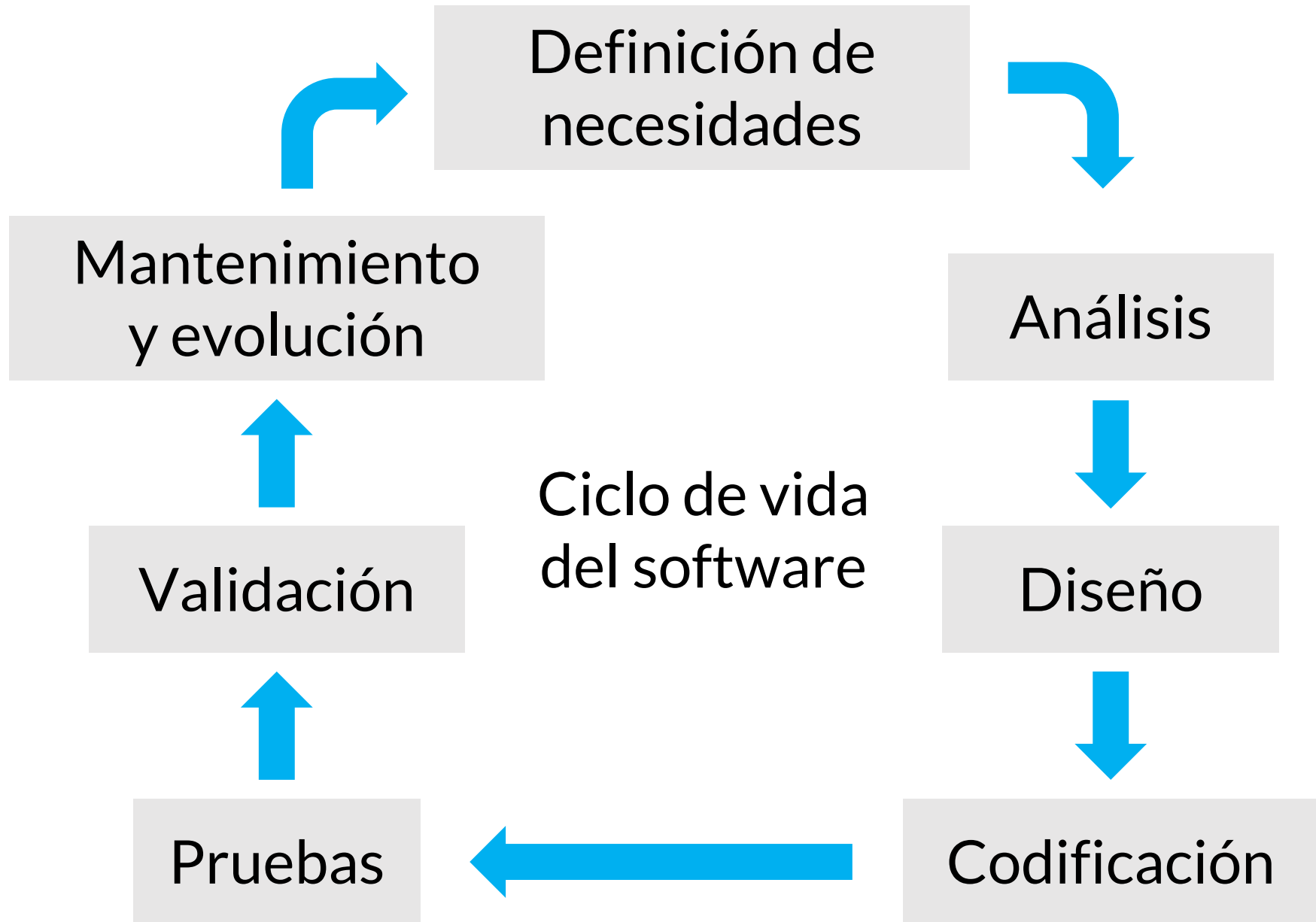
RECURSOS



TIEMPO

Actividades Clave de pruebas





ESTRATEGIA DE PRUEBAS



¿Que problema
tenemos actualmente?



¿O qué problemas
debemos evitar?

Escenarios y Contextos

- Seguridad
- Arquitectura
- Performance
- Usabilidad
- Escalabilidad

TESTING EN DESARROLLO DE SOFTWARE



TESTING

VS



CHECKING

ESTRATEGIAS DE CHECKING

- Solo se ejecutan si...
- Se ejecutan cada que...
- Se ejecutan de manera programada

ERRORES COMUNES DURANTE LA EJECUCIÓN

- Pruebas duplicadas

- Pruebas similares

- Pruebas sin valor agregado

- Pruebas caducadas

La automatización de pruebas consiste en el uso de **software especial** para controlar la ejecución de pruebas y la comparación entre los resultados obtenidos y los resultados esperados. Sin embargo, se trata de un **checking repetitivo y automatizado**.

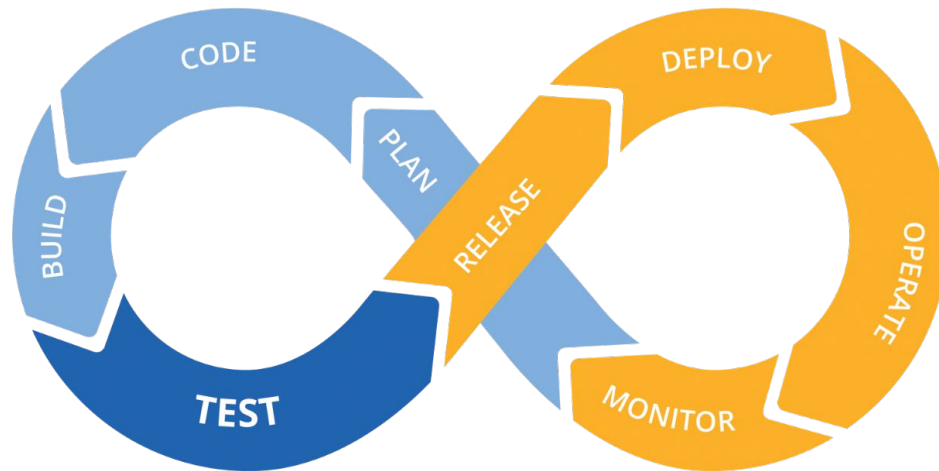
**DESVENTAJAS
CHECKING
MAL
EMPLEADO**

- Pobre cobertura de pruebas
- Falta de actualización
- Mal manejo de versiones

**VENTAJAS
CHECKING
BIEN
EMPLEADO**

- Correr pruebas en paralelo o en múltiples plataformas
- Reducción de error humano
- Probar grandes cantidades de datos

Por otro lado, cuando ya queremos hablar de **Integración continua y Liberación Continua**, entonces la **automatización** es la solución definitiva para la eficiencia del equipo de desarrollo digital y equipos DevOps



TESTING ÁGIL

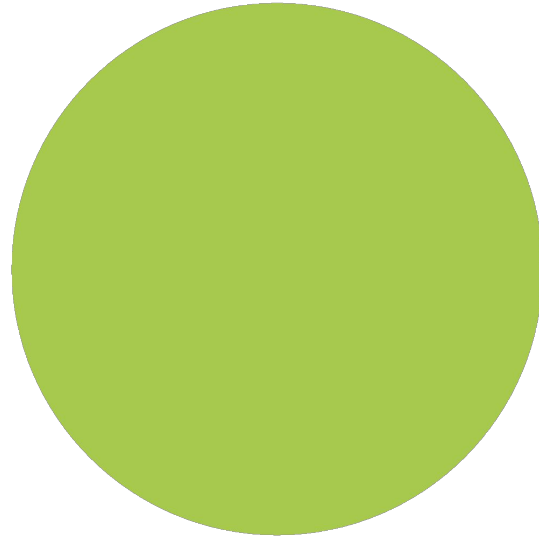
Testing ágil involucra a todos los miembros de un equipo ágil multifuncional, en el cual el rol del tester es el de un experto multifuncional, que garantiza se entregue el valor de negocio deseado al cliente a un ritmo sostenible y continuo.



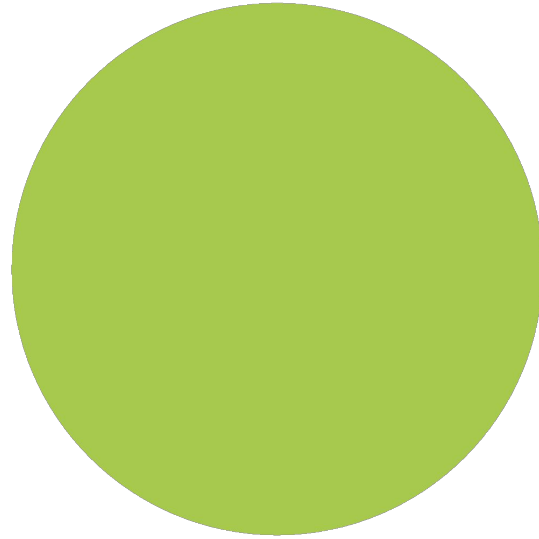
ESTRATEGIAS ÁGILES

-
- El Testing es de “todo el equipo”
 - El Testing puede ser independiente (opcional)
 - Integración continua
 - Testing guiado por pruebas (Test Driven Development – TDD)
 - Desarrollo guiado por comportamiento (Behaviour Driven Development – BDD)
 - Desarrollo guiado por pruebas de aceptación (Acceptance Test Driven Development – ATDD)

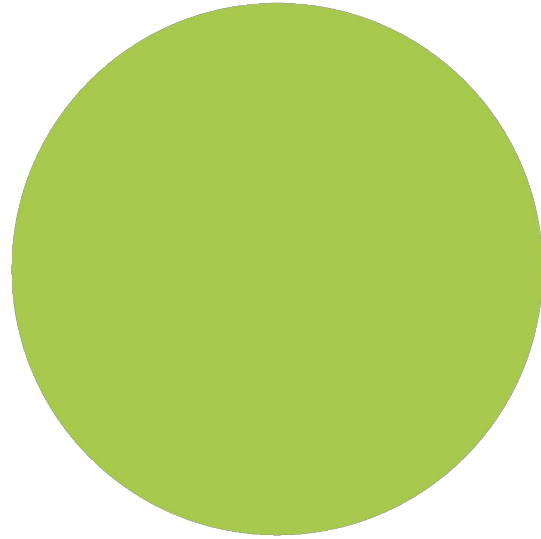
NIVELES DE PRUEBAS DE SOFTWARE



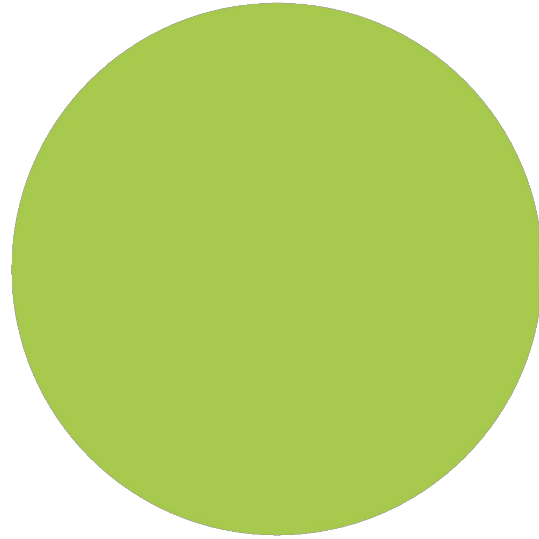
Prueba de
Componentes



Prueba de
Integración



Prueba de Sistema



Prueba de
Aceptación

TIPOS DE PRUEBAS DE SOFTWARE

1. PRUEBAS FUNCIONALES



Se entiende como las Funcionalidades del Sistema cómo “lo que el sistema hace”.

2. PRUEBAS NO-FUNCIONALES



El objetivo de esta es probar “como funciona el sistema”.

3. PRUEBAS ESTRUCTURALES



Para poder llevar a cabo estas pruebas, normalmente el tester debe tener conocimientos acerca de la tecnología y el stack que se está empleando

4. PRUEBAS DE MANEJO DE CAMBIOS



Es probar nuevamente un componente ya probado para verificar que no ha sido impactado por actualizaciones.